📖 Private 5G Network Cookbook & Blueprint

Standalone 5G Architecture: Open5GS Core + srsRAN gNodeB + USRP X310 + Multi-Slice Data Plane & Isolated MEC

Property	Details
Document Type	Operational Cookbook & Technical Blueprint
Base Architecture	Open5GS Core (v2.8.0), srsRAN Project gNodeB (v25.10.0)
RF Front-End	Ettus Research USRP X310 (HG FPGA, Band n78 / 3.5 GHz)
Data Plane Model	Multi-Namespace Linux Slicing with Strict VRF MEC Isolation
Workspace Directory	`/home/private-5g/Private-5g_setup/`
Status	Production Validated ✅

Synthesized from Private_5G_setup.pdf (by K. Nagasai) and the advanced multi-slice digital-twin testbed implementations.

---

1. 🏗️ Architecture & Network Topology

This cookbook details the deployment of a fully isolated, production-grade Standalone (SA) Private 5G Network. To ensure strict quality of service, security boundaries, and separation of concerns, the deployment organizes the 5G system across four dedicated Linux network namespaces and the Host OS control plane.

Network Domains & Responsibilities

Domain / Namespace	Component	Interface & Subnets	Functional Role
Host OS	Open5GS Control Plane	`veth-ran-host: 10.200.1.1/24` `veth-upf1-host: 10.200.3.1/24` `veth-upf2-host: 10.200.4.1/24`	AMF (NGAP: 38412), SMF (PFCP: 8805), NRF, SCP, AUSF, UDM, UDR, PCF. Manages subscribers, policy, and routing.
`ueran-ns`	srsRAN gNodeB	`veth-ran-ns: 10.200.1.2/24` `eno1: 192.168.10.1/24 (MTU 9000)`	Hosts base station (gNB), isolates physical Ethernet interface to USRP X310 SDR radio (`192.168.10.2`).
`upf1-ns`	Open5GS UPF1 (Internet)	`veth-upf1-ns: 10.200.3.2/24` `ogstun: 10.45.0.1/16`	Data Plane Slice 1 (SST 1, DNN: `internet`). Provides high-throughput general internet egress with Host NAT.
`upf2-ns`	Open5GS UPF2 (MEC Slice)	`veth-upf2-ns: 10.200.4.2/24`	Data Plane Slice 2 (SST 2, DNN: `critical` / `mec-secure` & `iot`). Dedicated MEC

		`ogstun: 10.46.0.1/16` `ogstun2: 10.47.0.1/16`	tunnel terminated on VRF Table 200.
`mec-app-ns`	MEC Applications	`veth-mec-ns: 192.168.50.2/24` `Gateway: 192.168.50.1`	Zero-Trust application domain (HTTP, RTSP video camera proxy, Flux controller). Accessible strictly by UPF2.

> *[!TIP]* ***Key Architecture Insight: Radio & Data Plane Isolation*** *By moving the physical Ethernet NIC (* `eno1` *) into* `ueran-ns` *, high-volume UDP sample streams between the USRP X310 and srsRAN are strictly confined to the radio namespace. Host network overhead and kernel packet processing for general OS tasks do not interfere with real-time SDR processing.*

---

2. 📻 Hardware & Radio Front-End Setup

1. USRP X310 Radio Connection

The USRP X310 is attached via direct Gigabit Ethernet (or 10G SFP+) to port `eno1` on the host server.

- **Host Interface:** `eno1` (assigned IP `192.168.10.1/24` inside `ueran-ns`)
- **USRP Device IP:** `192.168.10.2` (Default HG FPGA image)
- **Frame Size / MTU:** `mtu 9000` (Jumbo frames required to prevent packet overflow during 20 MHz / 30.72 Msps sample streaming)

2. Device Verification

```
# From Host OS before network setup:
uhd_find_devices
```

Expected Device Output: ```text

-- UHD Device 0

Device Address: serial: F5A867 addr: 192.168.10.2 fpga: HG product: X310 type: x300

```
### 3. Radio Parameters

| Parameter | Value | Rationale |
| :--- | :--- | :--- |
| **3GPP Band** | Band n78 (TDD) | Standard 3.5 GHz C-band for 5G Standalone
operation. Center Frequency: ~3489.42 MHz |
| **Downlink ARFCN** | `632628` | Standard 5G Center frequency configuration. |
| **Bandwidth** | 20 MHz | Requires 23.04 Msps sample rate, ideal for X310 over
1GbE/10GbE |
| **Subcarrier Spacing (SCS)** | 30 kHz | Standard 5G numerology (mu = 1) for Band n78
|
| **PLMN** | `00101` | Test network MCC: `001`, MNC: `01` (matches test USIMs) |
```

```
| **TAC** | `1` | Tracking Area Code (aligned with AMF and MongoDB profile) |

---

## 3. ⚙️ Configuration Specifications

### 1. Open5GS AMF (`configs/open5gs/amf.yaml`)

```yaml
logger:
  file:
    path: /var/log/open5gs/amf.log
    level: info

amf:
  sbi:
    server:
      - address: 127.0.0.5
        port: 7777
    client:
      scp:
        - uri: http://127.0.0.200:7777

  ngap:
    server:
      - address: 10.200.1.1 # NGAP server listens on Host side of veth-ran-host

  guami:
    - plmn_id:
        mcc: 001
        mnc: 01
      amf_id:
        region: 2
        set: 1

  tai:
    - plmn_id:
        mcc: 001
        mnc: 01
      tac: 1

  plmn_support:
    - plmn_id:
        mcc: 001
        mnc: 01
      s_nssai:
        - sst: 1
        - sst: 2
```

2. Open5GS SMF (`configs/open5gs/smf.yaml`)
```

```
smf:
  sbi:
    server:
      - address: 127.0.0.4
        port: 7777
      - address: 10.200.1.1
        port: 7777
    client:
      scp:
        - uri: http://127.0.0.200:7777

  pfcp:
    server:
      - address: 127.0.0.4
      - address: 10.200.1.1 # Host Gateway facing UPF Namespaces
    client:
      upf:
        - address: 10.200.3.2 # UPF1 (Internet Slice)
          dnn: internet
        - address: 10.200.4.2 # UPF2 (MEC Slice)
          dnn: [critical, mec-secure, iot]

  session:
    - subnet: 10.45.0.0/16
      gateway: 10.45.0.1
      dnn: internet
      dev: ogstun
    - subnet: 10.46.0.0/16
      gateway: 10.46.0.1
      dnn: critical
      dev: ogstun
    - subnet: 10.47.0.0/16
      gateway: 10.47.0.1
      dnn: iot
      dev: ogstun2
```

3. srsRAN 5G gNodeB (`configs/srsran/gnb_x310.yml`)

> *[!WARNING]* ***Important syntax note for srsRAN Project v25.10.0:*** *Tracking area configuration is derived directly from* `cell_cfg` *. Do not declare* `supported_tracking_areas` *under* `cu_cp` *.*

```
cu_cp:
  amf:
    addr: 10.200.1.1 # Host side IP of veth-ran-host
    port: 38412
    bind_addr: 10.200.1.2 # Namespace side IP of veth-ran-ns

ru_sdr:
  device_driver: uhd
  device_args: type=x300,addr=192.168.10.2,num_recv_frames=64,num_send_frames=64
```

```
  srate: 23.04
  otw_format: sc16
  tx_gain: 30
  rx_gain: 30

cell_cfg:
  dl_arfcn: 632628 # Band n78 (3.5 GHz)
  band: 78
  channel_bandwidth_MHz: 20
  common_scs: 30
  plmn: "00101"
  tac: 1
  pci: 1

log:
  filename: /tmp/gnb.log
  all_level: info
```

---

4. 📝 Step-by-Step Execution Recipes

Recipe 1: Initialize Network Namespaces & Hardware Isolation

Command: `sudo ./scripts/1_setup_network.sh` **Internal Actions:**

1. Cleans up any stale network namespaces and dangling veth links.
2. Creates `ueran-ns` and links it to Host with `veth-ran-host (10.200.1.1) ↔ veth-ran-ns (10.200.1.2)`.
3. Moves physical NIC `eno1` into `ueran-ns`, assigns `192.168.10.1/24`, sets MTU `9000`, and enables the link.
4. Creates `upf1-ns` (`10.200.3.2`) and `upf2-ns` (`10.200.4.2`).
5. Disables TCP/UDP checksum offloading on veth interfaces to avoid kernel dropping namespace-traversing packets.
6. Adds routes on Host for UE subnets: `10.45.0.0/16` via `10.200.3.2` and `10.46.0.0/16` via `10.200.4.2`.
7. Enables `net.ipv4.ip_forward=1` and adds iptables MASQUERADE on the active WAN interface (`wlp0s20f3`).

Recipe 2: Start 5G Core Control Plane

Command: `sudo ./scripts/start_core.sh` **Internal Actions:**

1. Disables and stops the Host systemd `open5gs-upfd` daemon to eliminate port conflicts.
2. Copies `amf.yaml` and `smf.yaml` templates to `/etc/open5gs/`.
3. Restarts Open5GS control plane services (`open5gs-amfd`, `open5gs-smfd`, `open5gs-nrfd`, etc.).
4. Verifies AMF NGAP socket (`10.200.1.1:38412`) and SMF PFCP socket (`10.200.1.1:8805`).

Recipe 3: Launch Dual UPF Data Planes

Command: `sudo ./scripts/start_upfs.sh` **Internal Actions:**

1. Enables IP forwarding and disables reverse path filtering inside `upf1-ns` and `upf2-ns`.
2. Creates TUN interface `ogstun` (`10.45.0.1/16`) in `upf1-ns`.

3. Creates TUN interfaces `ogstun` (`10.46.0.1/16`) and `ogstun2` (`10.47.0.1/16`) in `upf2-ns` .
4. Starts `open5gs-upfd` in `upf1-ns` using `upf_internet.yaml` .
5. Starts `open5gs-upfd` in `upf2-ns` using `upf_mec.yaml` .
6. Displays cleanly formatted status with active daemon PIDs.

Recipe 4: Establish Zero-Trust MEC Domain

Command: `sudo ./setup_mec.sh` **Internal Actions:**

1. Creates `mec-app-ns` with `veth-mec-host (192.168.50.1) ↔ veth-mec-ns (192.168.50.2)` .
2. Deletes `192.168.50.0/24` from the Host main routing table (making MEC invisible to OS and UPF1).
3. Inserts route `192.168.50.0/24 dev veth-mec-host` into custom Routing Table 200.
4. Adds return route for MEC slice: `10.46.0.0/16 via 10.200.4.2 dev veth-upf2-host table 200` .
5. Enforces Policy Rules: packets arriving on `veth-upf2-host` and `veth-mec-host` must use Table 200.

Recipe 5: Launch srsRAN 5G gNodeB

Command: `sudo ./scripts/start_gnb.sh` **Internal Actions:**

1. Verifies `eno1` is inside `ueran-ns` and pings USRP X310 (`192.168.10.2`).
2. Verifies NGAP link to AMF (`10.200.1.1:38412`).
3. Copies `gnb_x310.yml` to `/opt/srsRAN_gnb/gnb_config.yml` (bypassing AppArmor restrictions).
4. Launches `/usr/local/bin/gnb` inside `ueran-ns` , broadcasting cell on Band n78.

---

5. 🛡️ Zero-Trust MEC Isolation & Ping Testing Theory

1. Packet Traversal from 5G UE to MEC Server

When a connected 5G smartphone requests a MEC service (e.g. `curl http://192.168.50.2:8000/`):

1. **UE (`10.46.x.x`)**: Sends IP packet over 5G NR RF air interface to USRP X310.
2. **srsRAN gNB (`ueran-ns`)**: Encapsulates user packet into a 3GPP GTP-U UDP tunnel (port 2152) and transmits it to UPF2 (`10.200.4.2`).
3. **UPF2 (`upf2-ns`)**: Decapsulates the GTP-U packet and outputs raw IP onto TUN interface `ogstun` .
4. **UPF2 Gateway Routing**: Kernel forwards packet via default route `10.200.4.1 dev veth-upf2-ns` towards the Host.
5. **Host Policy Routing**: Packet arrives on Host interface `veth-upf2-host` . Policy rule `iif veth-upf2-host lookup 200` matches! Table 200 routes the packet into `veth-mec-host` .
6. **MEC Application (`mec-app-ns`)**: Receives request on `192.168.50.2` and generates the HTTP response.
7. **Return Path**: Response arrives on Host from `veth-mec-host` , matches rule `iif veth-mec-host lookup 200` , and Table 200 routes back to `10.46.0.0/16 via 10.200.4.2 dev veth-upf2-host` into UPF2.

2. The `ping -I ogstun` Fallacy: Device vs. Address Binding

> *[!CAUTION]* ***Why `ping -I ogstun 192.168.50.2` Fails with 100% Packet Loss*** *In Linux, passing an interface name (`-I ogstun`) calls `setsockopt(SO_BINDTODEVICE)` . This instructs the kernel to bypass normal routing table lookups and force packets directly into the netdev transmit queue of `ogstun` .*

> *Because `ogstun` is a TUN device owned by `open5gs-upfd`, the UPF reads the packet and attempts to match destination `192.168.50.2` against active UE downlink PDU sessions. Because the MEC server is an external host and not a connected UE, the UPF discards the packet.*

The Correct Syntax: Pass the source IP address rather than the device name:

```
sudo ip netns exec upf2-ns ping -I 10.46.0.1 192.168.50.2
```

When an IP is provided, the kernel uses standard routing out `veth-upf2-ns`, successfully reaching Table 200.

3. Security Isolation Verification Matrix

```
# 1. From UPF2 (Authorized MEC Slice) -> MUST SUCCEED (0% loss)
sudo ip netns exec upf2-ns ping -c 3 192.168.50.2
sudo ip netns exec upf2-ns curl http://192.168.50.2:8000/

# 2. From Host OS -> MUST FAIL (Destination Host Unreachable / 100% loss)
ping -c 2 192.168.50.2

# 3. From ueran-ns (gNB Base Station) -> MUST FAIL (100% loss)
sudo ip netns exec ueran-ns ping -c 2 192.168.50.2

# 4. From UPF1 (Internet Slice) -> MUST FAIL (100% loss)
sudo ip netns exec upf1-ns ping -c 2 192.168.50.2
```

---

6. 👤 Subscriber Provisioning & Verification

1. WebUI Provisioning

1. Access Open5GS WebUI at: `http://localhost:9999` (Default credentials: `admin` / `1423`).
2. Click **Subscribers** → **Add Subscriber**.
3. Enter SIM parameters:
   - **IMSI**: `001010010001566` (or your physical USIM IMSI)
   - **Secret Key (K)**: Enter 128-bit key in Hex
   - **Operator Code (OPc)**: Enter 128-bit OPc in Hex
4. Add Network Slices:
   - **Slice 1**: SST: `1`, Default: Checked, Session DNN: `internet`
   - **Slice 2**: SST: `2`, Default: Unchecked, Session DNN: `critical`
5. Save subscriber profile. Changes take immediate effect via MongoDB.

2. MongoDB Query

View active subscribers and provisioned slices from CLI:

```
mongosh open5gs --eval "db.subscribers.find({}, {imsi: 1, 'slice.sst': 1,
'slice.session.name': 1})"
```

---

7. 🚑 Troubleshooting Reference & Operational FAQs

Observed Issue	Root Cause	Resolution
INI was not able to parse cu_cp.supported_tracking_areas	srsRAN v25.10 expects PLMN and TAC under `cell_cfg`, not directly under `cu_cp`. CU-CP automatically derives supported TAs from the cell definition.	Move `plmn` and `tac` into `cell_cfg`.
socket bind(2) [10.200.1.1]:8805 failed: Cannot assign requested address	SMF attempted to start before `veth-ran-host` was created by the network setup script.	Execute `sudo ./scripts/1_setup_network.sh` before starting the Core control plane.
uhd_find_devices reports No Devices Found	USRP X310 power is OFF, cable is unplugged, or `eno1` was moved into `ueran-ns`.	If `eno1` is in namespace, query inside namespace: `sudo ip netns exec ueran-ns uhd_find_devices`.
No Carrier / Link Down on eno1	Physical link negotiation issue with USRP X310 SFP/GigE port.	Power-cycle USRP X310 and verify link light is solid amber/green. Run `sudo ethtool eno1`.
Phone connects but cannot browse internet	WAN interface NAT MASQUERADE not applied or wrong interface specified.	Verify WAN interface in `1_setup_network.sh` is `wlp0s20f3` and `net.ipv4.ip_forward=1`.